Job Board Backend - PowerShell Smoke Test Commands

Run these from `jobboard-project/backend` in PowerShell.

Start the Dev Server

```powershell
# from backend/
npm run dev
```

Expected Output:

```
MongoDB connected
Server running on port 4000
```

Health Check

```powershell
curl.exe -i http://localhost:4000/health
```

Expected Output:

```
HTTP/1.1 200 OK
OK
```

Authentication Flows

Register a Candidate

```powershell
```

```
$body = @{
    name = "Test Candidate"
    email = "cand@example.com"
    password = "secret123"
    role = "candidate"
} | ConvertTo-Json

Invoke-RestMethod -Uri "http://localhost:4000/api/auth/register" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body
```

Expected: Token present, user object with `role: "candidate"`

Register an Employer

```powershell
$body = @{
    name = "Test Employer"
    email = "emp@example.com"
    password = "secret123"
    role = "employer"
} | ConvertTo-Json

Invoke-RestMethod -Uri "http://localhost:4000/api/auth/register" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body
```

Login (Employer)

```powershell
```

```
$body = @{
    email = "emp@example.com"
    password = "secret123"
} | ConvertTo-Json

$response = Invoke-RestMethod -Uri "http://localhost:4000/api/auth/login" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body

# Save token
$EMP_TOKEN = $response.token
Write-Host "Employer Token: $EMP_TOKEN"
```

Login (Candidate)

```powershell
$body = @{
    email = "cand@example.com"
    password = "secret123"
} | ConvertTo-Json

$response = Invoke-RestMethod -Uri "http://localhost:4000/api/auth/login" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body

# Save token
$CAND_TOKEN = $response.token
Write-Host "Candidate Token: $CAND_TOKEN"
```

Check /me Endpoint

```powershell
$headers = @{
    Authorization = "Bearer $EMP_TOKEN"
}

Invoke-RestMethod -Uri "http://localhost:4000/api/auth/me" `
    -Method Get `
    -Headers $headers
```

Expected: User object with email and role

Job Endpoints

Create a Job (Employer Only)

```powershell
$headers = @{
    Authorization = "Bearer $EMP_TOKEN"
    "Content-Type" = "application/json"
}

$body = @{
    title = "Backend Engineer"
    description = "Do great things"
    location = "Remote"
    remote = $true
    salaryMin = 60000
    salaryMax = 120000
    tags = @("node", "typescript")
} | ConvertTo-Json

$jobResponse = Invoke-RestMethod -Uri "http://localhost:4000/api/jobs" `
    -Method Post `
    -Headers $headers `
    -Body $body

# Save job ID
$JOB_ID = $jobResponse._id
Write-Host "Job ID: $JOB_ID"
```

Expected: 201 response, job object with `_id`

List Jobs (Public)

```powershell
Invoke-RestMethod -Uri "http://localhost:4000/api/jobs?q=Backend&page=1&limit=10&sort=date" `
    -Method Get
```

Expected: JSON with `total`, `page`, `pages`, `data` array

Get Job Details

```powershell
Invoke-RestMethod -Uri "http://localhost:4000/api/jobs/$JOB_ID" `
    -Method Get
```

Upload (S3 Presign) Flow

Request Presigned URL (Candidate)

```powershell
$headers = @{
    Authorization = "Bearer $CAND_TOKEN"
}

$uri = "http://localhost:4000/api/uploads/presign?filename=resume.pdf&contentType=application/pdf"

$presignResponse = Invoke-RestMethod -Uri $uri `
    -Method Get `
    -Headers $headers

# Save URLs
$UPLOAD_URL = $presignResponse.uploadUrl
$PUBLIC_URL = $presignResponse.publicUrl
$FILE_KEY = $presignResponse.fileKey

Write-Host "Upload URL: $UPLOAD_URL"
Write-Host "Public URL: $PUBLIC_URL"
```

Expected: `uploadUrl`, `fileKey`, `publicUrl` returned

Simulate Upload to S3

```powershell
# Using Invoke-RestMethod
$filePath = ".\test-resume.pdf"
$fileBytes = [System.IO.File]::ReadAllBytes($filePath)

Invoke-RestMethod -Uri $UPLOAD_URL `
    -Method Put `
    -ContentType "application/pdf" `
    -Body $fileBytes
```

Expected: 200 or 201 success from S3

Application Workflow

Create Application (Candidate)

```powershell
$headers = @{
    Authorization = "Bearer $CAND_TOKEN"
    "Content-Type" = "application/json"
}

$body = @{
    jobId = $JOB_ID
    resumeUrl = $PUBLIC_URL
    originalFileName = "resume.pdf"
    fileSize = 12345
} | ConvertTo-Json

$appResponse = Invoke-RestMethod -Uri "http://localhost:4000/api/applications" `
    -Method Post `
    -Headers $headers `
    -Body $body

# Save application ID
$APP_ID = $appResponse._id
Write-Host "Application ID: $APP_ID"
```

Expected: 201 application object, employer receives email

Candidate Views Applications

```powershell
$headers = @{
    Authorization = "Bearer $CAND_TOKEN"
}

Invoke-RestMethod -Uri "http://localhost:4000/api/applications" `
    -Method Get `
    -Headers $headers
```

Expected: Array containing the created application

Employer Updates Status

```powershell
```

```
$headers = @{
    Authorization = "Bearer $EMP_TOKEN"
    "Content-Type" = "application/json"
}

$body = @{
    status = "Screening"
} | ConvertTo-Json

Invoke-RestMethod -Uri "http://localhost:4000/api/applications/$APP_ID/status" `
    -Method Put `
    -Headers $headers `
    -Body $body
```

Expected: Updated application with new status, candidate receives email

Test Invalid Transition

```powershell
# First set to Rejected
$body = @{
    status = "Rejected"
} | ConvertTo-Json

Invoke-RestMethod -Uri "http://localhost:4000/api/applications/$APP_ID/status" `
    -Method Put `
    -Headers $headers `
    -Body $body

# Then try to set to Offer (should fail)
$body = @{
    status = "Offer"
} | ConvertTo-Json

try {
    Invoke-RestMethod -Uri "http://localhost:4000/api/applications/$APP_ID/status" `
        -Method Put `
        -Headers $headers `
        -Body $body
} catch {
    Write-Host "Expected error: Invalid status transition"
    $_.Exception.Message
}
```

Expected: 400 error with "Invalid status transition"

Validation & Error Handling

Bad Register Payload

```powershell
$body = @{
    email = "not-an-email"
} | ConvertTo-Json

try {
    Invoke-RestMethod -Uri "http://localhost:4000/api/auth/register" `
        -Method Post `
        -ContentType "application/json" `
        -Body $body
} catch {
    Write-Host "Expected validation error"
    $_.Exception.Message
}
```

Expected: 400 validation error

Bad Presign Request (Invalid File Type)

```powershell
$headers = @{
    Authorization = "Bearer $CAND_TOKEN"
}

$uri = "http://localhost:4000/api/uploads/presign?filename=resume.docx&contentType=application/msword"

try {
    Invoke-RestMethod -Uri $uri `
        -Method Get `
        -Headers $headers
} catch {
    Write-Host "Expected error: Only PDF files allowed"
    $_.Exception.Message
}
```

Expected: 400 with "Only PDF files allowed"

Automated Checks

TypeScript Type Check

```powershell
npx tsc --noEmit
```

Expected: No type errors

Lint

```powershell
npm run lint
```

Run Tests

```powershell
npm test
# or
npx jest
```

Complete Test Script

Save this as `test-api.ps1` and run with `.\test-api.ps1`:

```powershell
```

```
# Complete Job Board API Test Script

Write-Host "=== Starting Job Board API Tests ===" -ForegroundColor Green

# 1. Register Candidate
Write-Host "`n1. Registering Candidate..." -ForegroundColor Yellow
$body = @{
    name = "Test Candidate"
    email = "cand@example.com"
    password = "secret123"
    role = "candidate"
} | ConvertTo-Json

Invoke-RestMethod -Uri "http://localhost:4000/api/auth/register" -Method Post -ContentType "application/json" -Body $body

# 2. Register Employer
Write-Host "`n2. Registering Employer..." -ForegroundColor Yellow
$body = @{
    name = "Test Employer"
    email = "emp@example.com"
    password = "secret123"
    role = "employer"
} | ConvertTo-Json

Invoke-RestMethod -Uri "http://localhost:4000/api/auth/register" -Method Post -ContentType "application/json" -Body $body

# 3. Login Employer
Write-Host "`n3. Logging in Employer..." -ForegroundColor Yellow
$body = @{
    email = "emp@example.com"
    password = "secret123"
} | ConvertTo-Json

$response = Invoke-RestMethod -Uri "http://localhost:4000/api/auth/login" -Method Post -ContentType "application/json" -B
$EMP_TOKEN = $response.token
Write-Host "✓ Employer Token obtained" -ForegroundColor Green

# 4. Login Candidate
Write-Host "`n4. Logging in Candidate..." -ForegroundColor Yellow
$body = @{
    email = "cand@example.com"
    password = "secret123"
} | ConvertTo-Json

$response = Invoke-RestMethod -Uri "http://localhost:4000/api/auth/login" -Method Post -ContentType "application/json" -B
$CAND_TOKEN = $response.token
```

```
Write-Host "✓ Candidate Token obtained" -ForegroundColor Green

# 5. Create Job
Write-Host "`n5. Creating Job..." -ForegroundColor Yellow
$headers = @{
   Authorization = "Bearer $EMP_TOKEN"
}
$body = @{
   title = "Backend Engineer"
   description = "Do great things"
   location = "Remote"
   remote = $true
   salaryMin = 60000
   salaryMax = 120000
   tags = @("node", "typescript")
} | ConvertTo-Json

$jobResponse = Invoke-RestMethod -Uri "http://localhost:4000/api/jobs" -Method Post -Headers $headers -ContentType "app
$JOB_ID = $jobResponse._id
Write-Host "✓ Job created: $JOB_ID" -ForegroundColor Green

# 6. List Jobs
Write-Host "`n6. Listing Jobs..." -ForegroundColor Yellow
Invoke-RestMethod -Uri "http://localhost:4000/api/jobs?q=Backend&page=1&limit=10" -Method Get
Write-Host "✓ Jobs listed successfully" -ForegroundColor Green

# 7. Request Presigned URL
Write-Host "`n7. Requesting Presigned URL..." -ForegroundColor Yellow
$headers = @{
   Authorization = "Bearer $CAND_TOKEN"
}
$presignResponse = Invoke-RestMethod -Uri "http://localhost:4000/api/uploads/presign?filename=resume.pdf&contentType=
$PUBLIC_URL = $presignResponse.publicUrl
Write-Host "✓ Presigned URL obtained" -ForegroundColor Green

# 8. Create Application
Write-Host "`n8. Creating Application..." -ForegroundColor Yellow
$headers = @{
   Authorization = "Bearer $CAND_TOKEN"
}
$body = @{
   jobId = $JOB_ID
   resumeUrl = $PUBLIC_URL
   originalFileName = "resume.pdf"
   fileSize = 12345
} | ConvertTo-Json
```

```
$appResponse = Invoke-RestMethod -Uri "http://localhost:4000/api/applications" -Method Post -Headers $headers -ContentT
$APP_ID = $appResponse._id
Write-Host "✓ Application created: $APP_ID" -ForegroundColor Green

# 9. Update Application Status
Write-Host "`n9. Updating Application Status..." -ForegroundColor Yellow
$headers = @{
    Authorization = "Bearer $EMP_TOKEN"
}
$body = @{
    status = "Screening"
} | ConvertTo-Json

Invoke-RestMethod -Uri "http://localhost:4000/api/applications/$APP_ID/status" -Method Put -Headers $headers -ContentTy
Write-Host "✓ Application status updated" -ForegroundColor Green

Write-Host "`n=== All Tests Completed ===" -ForegroundColor Green
```

Notes

- Use `Invoke-RestMethod` for JSON responses (auto-parses)
- Use `curl.exe` for raw HTTP responses (headers, status codes)
- PowerShell variables use `$` prefix
- `@{}` creates hashtables, `ConvertTo-Json` converts to JSON
- Wrap API calls in `try/catch` for error handling